

Title: SmartCare Appointment and Wellness System (SAWS)

Team Size: ~6 Members

Project Length: Approx. 2.5 months

Project Leader: TA - Prashant Rawat



Objective:

The primary objective is to build a cloud data plumbing system designed to manage and automate healthcare appointment and wellness support services. The application must use serverless technologies to process on-demand data and should be easy for various client types to use. Students will build a cloud-native multi-cloud application utilizing backend-as-a-service (BaaS), event-driven architecture, serverless computing, and cloud analytics.

Plagiarism Policy:

- This project is a group task. Collaboration outside the assigned group amounts to a violation of the academic integrity policy and will be reported.
- Content cannot be copied verbatim from any source(s). Understand the concept and write in your own words while citing sources properly.
- AI usage policy must follow the course syllabus guidelines.
- The university academic integrity policy applies to all submitted materials.

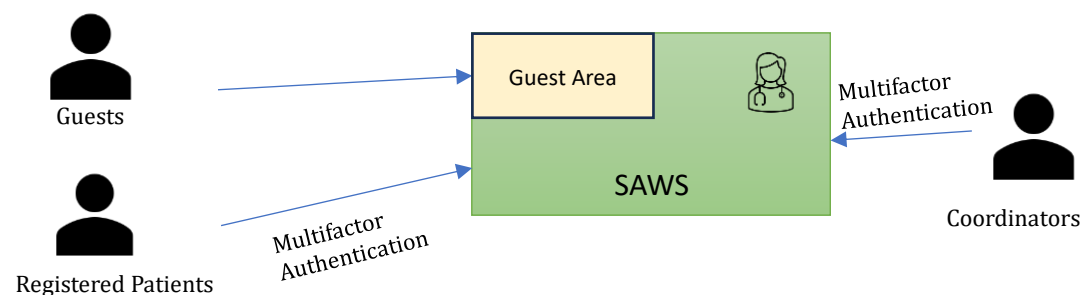
Hypothetical Scenario:

HealthBridge Technologies is developing a serverless *SmartCare Appointment and Wellness System (SAWS)* using a multi-cloud deployment model and backend-as-a-service (BaaS).

The SAWS platform helps patients schedule healthcare consultations, wellness sessions, and follow-up appointments while enabling healthcare coordinators to manage requests efficiently.

The system must provide:

- Customized services for registered patients
- Administrative features for wellness coordinators
- Limited functionality for guest users
- A virtual assistant for support and navigation
- Asynchronous communication between patients and coordinator



saurabh.dey@dal.ca

This content is protected and may not be shared, uploaded, or distributed.

Use Case: SAWS User Types

There will be three types of users in the WinterCloud Dispatch System:

1. Guests:

- Visit the application to check healthcare and wellness services availability
- View available doctors, wellness sessions, and consultation timings
- Use the virtual assistant/chatbot for navigation assistance
- View overall feedback and sentiment analysis for healthcare services
- Explore general wellness packages and service charges

2. Registered Patients:

Registered Patients can:

- Perform all guest operations
- Receive notifications upon successful registration and sign-in
- Access the system using sequential multi-stage authentication:
 - 1st Stage: UserID/ Password
 - 2nd Stage: Security Question/Answer
 - 3rd Stage: Healthcare code clue using Caesar cipher
- Book healthcare consultations or wellness sessions for specific dates and times
- View appointment history and appointment status
- Use the virtual assistant to:
 - Navigate the application
 - Retrieve appointment reference codes
 - Check appointment details
 - Submit concerns or support requests
- Provide feedback using a structured feedback form
- Receive appointment confirmation and reminder notifications

3. Wellness Coordinators (Admin Users):

Wellness Coordinators can:

- Perform all guest operations
- Receive notifications upon successful registration and sign-in
- Access the platform using the same sequential multi-factor authentication
- Add/update healthcare and wellness services
- Manage doctors/specialists schedules
- Approve or reject appointment requests
- Manage service pricing and promotional packages
- Use the virtual assistant to retrieve appointment details
- Communicate asynchronously with patients after support requests are submitted
- Monitor platform usage analytics and patient engagement

TASK DETAILS

Module Details:

You must use **Serverless architecture** to build the following modules:

1. User Management & Authentication Module:

Implement sign-up validation and multi-factor authentication using a light-weight front-end application and backend services.

- User details stored in **DynamoDB** (or equivalent NoSQL database).
- 1st stage (User ID/Password) handled by **AWS Cognito**.
- 2nd stage (Question/Answer) handled by **DynamoDB + AWS Lambda**.
- 3rd stage (Caesar cipher) handled by **AWS Lambda + DynamoDB**.

Expected Features:

- Registration validation
- Secure login
- Session management
- Role-based access

2. Virtual Assistant Module (Chatbot):

Bots must respond to queries. (Use AWS Lex + DynamoDB + AWS Lambda) **OR** (GCP DialogueFlow + Firestore + Cloud Function)

Functionalities:

- Application navigation assistance
- Appointment lookup using appointment reference code
- Wellness package inquiry support
- Accept patient concerns and forward them to coordinators
- Frequently asked questions (FAQ) support

3. Message Passing Module:

Used for message passing between parties (GCP pub/sub).

Functionalities:

- Patients submit support concerns
- Subscriber functions forward requests to a random coordinator
- Communication logs stored in Firestore or DynamoDB
- Support asynchronous communication

Additional Feature:

- Real-time synchronous chat patient $\leftrightarrow$ Wellness coordinator

4. Notifications Module: Use AWS SNS and AWS SQS.

Notifications Required:

- Successful registration notification
- Successful login notification
- Appointment confirmation notification
- Appointment cancellation notification
- Appointment reminder notification

Example Workflow:

- Appointment request enters SQS queue
- Lambda validates request
- SNS sends notification to patient

5. Data Analysis & Visualization:

The output of this module will be visible to all user types.

Functionalities:

- Coordinators can view:
 - Total registered patients
 - Login statistics
 - Appointment trends
 - Popular wellness services
- Dashboard integration using:
 - Looker Studio **and/or** QuickSight

Feedback Analysis:

- Patients submit service feedback
- Backend automatically performs sentiment analysis using:
 - AWS Comprehend **OR**
 - Google Natural Language API
- Sentiment results displayed in frontend

Visualization Requirements:

- Feedback table
- Charts for appointment statistics
- Service popularity visualization

6. Web Application Building and Deployment:

Requirements:

- Build frontend using React or similar framework
- Frontend should invoke all backend services
- Deployment using:
 - AWS Fargate
 - GCP Cloud Run
- Infrastructure automation using:
 - AWS CloudFormation
 - Terraform
 - GCP Deployment Manager

General Tasks:

Test cases must be validated and documented.

Required Testing:

- Successful authentication testing
- Failed login testing
- Database integrity testing
- Chatbot utterance testing
- API testing
- Lambda/Cloud Function event testing
- Queue/message testing
- Notification delivery testing

Evidence Required:

- Screenshots
- Logs
- Test case tables

Maintain code quality: focus on readability, reusability, cleaning, and commenting.

Perform regular **git commits**, expected mainly during Sprint 2 and Sprint 3.

Use CS gitlab as repository and project management (e.g., issue board, iterations, task management).

PROJECT REQUIREMENT DOCUMENT – CSCI 5410/S26

Deliverables – Project (40% of Course Grade)

Project requirement details will be shared later.

This component requires developing a group project. Regular project status update is required. Project documentations, and deliverables are required at different stages.

You need to maintain DAL FCS gitlab code repository

Sprint 1 (25% of Project Grade): Planning Phase

Work Completed (10%)	Report (10%) – max 4 pages	Peer Review (5%)
Sprint 1 - Proof of work must be demonstrated through meaningful GitLab activity, including commits, uploaded research/planning documents, meeting notes, architecture drafts, task boards/issues, references to cloud service documentation, API investigation notes, and evidence of collaborative contribution from team members.	The Sprint 1 report should document the foundational planning and research activities completed by the team. The report must clearly describe the project idea, problem statement, objectives, intended users, and proposed system scope. Teams should include preliminary architecture/design concepts, selected AWS/GCP services with justification, initial API and technology research, and a summary of cloud documentation/manuals reviewed during the sprint.	Each member will document activities of other members. E.g. • Involvement in reviewing cloud documentation, APIs, and technical resources. • Quality and frequency of GitLab contributions (commits, issues, documentation, updates). • Collaboration and communication with team members • Attendance and participation in team meetings/discussions • Initiative in proposing ideas, architecture, or technical solutions

Sprint 2 (25% of Project Grade): Development & Testing Phase

Work Completed (10%)	Report (10%) – max 6 pages	Peer Review (5%)
Teams are expected to actively use GitLab Proof of work should include meaningful commits, architecture diagrams, prototype screenshots, API testing evidence, deployment attempts, configuration files, technical notes, and records of collaborative development activities. Sprint 2 focuses on major system implementation	The Sprint 2 report should document the technical design and prototype development progress. The report must include updated system architecture diagrams, database/ERD design, API structure, selected cloud services and configurations, user interface mockups/prototypes, and implementation progress completed during the sprint. Teams should also discuss technical challenges encountered, design decisions made, testing performed, deployment progress, and planned objectives for the next sprint. Screenshots, diagrams, GitLab activity summaries, and references to cloud services/APIs used should be included where appropriate.	Each member will document activities of other members. E.g. • Participation in frontend/backend prototype development • Contribution to database/API/cloud configuration tasks • Quality and consistency of GitLab commits and updates • Collaboration and communication during technical development • Participation in testing and troubleshooting activities

Sprint 3 (25% of Project Grade): Development & Testing Phase

Work Completed (10%)	Report (10%) – max 6 pages	Peer Review (5%)
Sprint 3 focuses on major system implementation and cloud service integration. Completion of Sprint 3 means the team has implemented the primary application features, integrated serverless/cloud services, connected APIs and databases, and established working communication between system components.	Teams should discuss report challenges faced during implementation, performance considerations, security measures, and adjustments made to the original design or sprint plan. Relevant screenshots, diagrams, testing evidence, GitLab summaries, and deployment references should be included. The report should include completed modules/features, cloud architecture updates, API integrations, database interactions, testing activities, deployment status, and evidence of working functionality.	Each member will document activities of other members. E.g. • Participation in frontend/backend prototype development • Contribution to database/API/cloud configuration tasks • Quality and consistency of GitLab commits and updates • Collaboration and communication during technical development

saurabh.dey@dal.ca

This content is protected and may not be shared, uploaded, or distributed.

PROJECT REQUIREMENT DOCUMENT – CSCI 5410/S26

Teams should demonstrate significant functional progress and begin performing integration and system testing. GitLab activity should clearly reflect ongoing development through commits, merge requests, issue tracking, testing evidence, deployment updates, and collaborative problem-solving activities.		• Participation in testing and troubleshooting activities • Professional conduct and accountability
---	--	--

Sprint 4 (25% of Project Grade): Closing Phase

Work Completed (10%) - DEMO	Report (10%)	Peer Review (5%)
GitLab repositories should contain complete project documentation, finalized code, deployment configurations, testing evidence A recorded DEMO is expected from all the members of each team.	The report should include screenshots, testing summaries, cloud deployment evidence, GitLab contribution summaries, references, and a reflection on team collaboration and project outcomes. Final presentation/demo preparation details may also be included.	Each member will document activities of other members. E.g. • Contribution toward final documentation and presentation preparation • Collaboration and communication during project finalization • Initiative in resolving remaining technical issues • Support provided to team members during completion phase

Project Expectations:

- Please respect every individual and follow the “Culture of Respect” section provided in the course syllabus.
- All members should contribute equally to the project.
- One or two member(s) of a team completing the project cannot be considered as a successful group project, and the entire group may receive lower score.
- All members of a team may not receive similar score. Each sprint will have individual contribution grade as well.

Architecture Components

Component	Required Technology
Authentication	AWS Cognito
Database	DynamoDB / Firestore
Backend Logic	AWS Lambda / Cloud Functions
Chatbot	AWS Lex / DialogFlow
Messaging	SNS/SQS / PubSub
Frontend	React
Deployment	Cloud Run / Fargate
Analytics	Looker Studio / QuickSight

saurabh.dey@dal.ca

This content is protected and may not be shared, uploaded, or distributed.